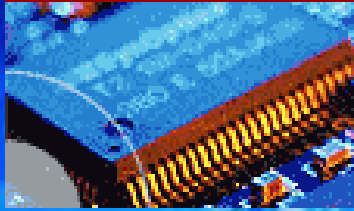
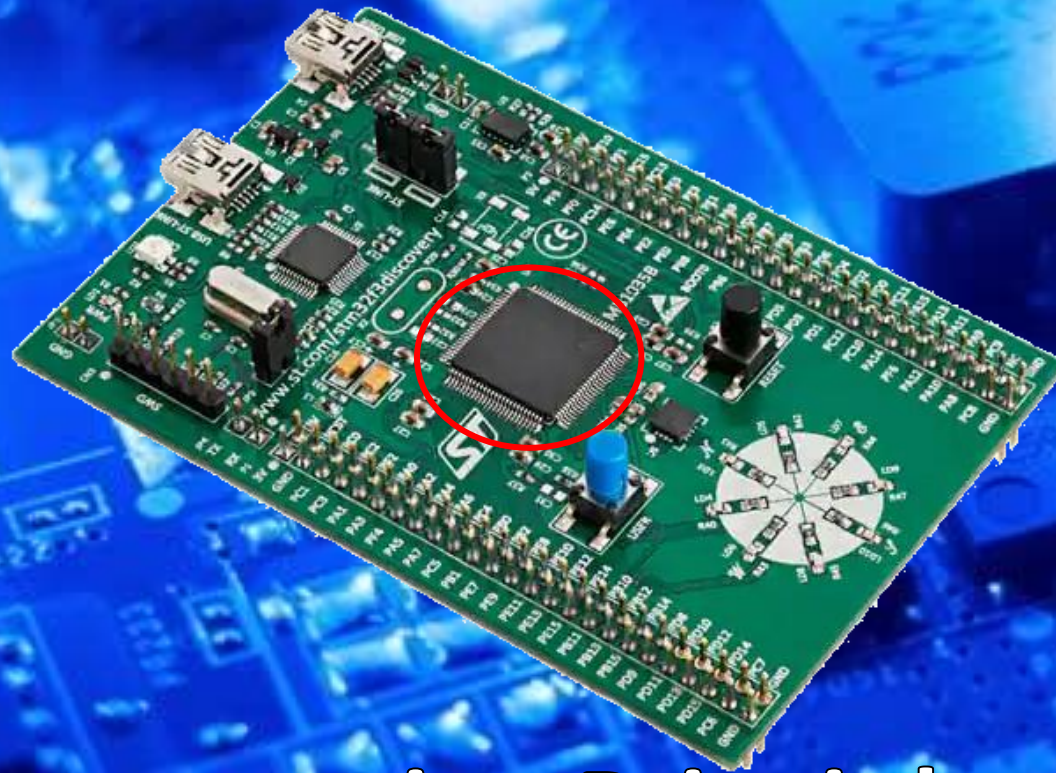


EEE3096S



**Embedded
Systems II**



I/O Programming Principles and Memory Addressed I/O Embedded Systems II

L11a

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Outline of Lecture

- Programming Memory Ports
- Memory Mapped & Port Mapped
- Use of the volatile keyword in C (recap)
- The Importance of Linkers
- Memory Mapped I/O
- Assessing understanding of peripheral registers

Embedded Systems Software Techniques

1. PERIPHERAL ACCESS & I/O

Focusing on **Addressed-Mapped I/O** in these slides

Lecture 12 involved port-mapped, with the PCP (Peripheral Controller Processor) activity (see “Lecture L12 - PCP Activity.pdf”)

Embedded Systems II

Accessing Hardware in C

- Two approaches for external device IO
 - Port Mapped IO (brief recap ... see Lect12)
 - Memory Mapped IO ← *we're been using!*

Port Mapped IO

reminder

- A special IO address space is used to access devices
- Special CPU instructions are used to access this address space (e.g. **IN**, **OUT**)

Advantage:

- Peripherals can contain simpler memory address decoding logic.

Disadvantage:

- Special instructions and programming techniques required to access peripherals

Either Harvard or von Neumann architectures can use port mapped IO

Example mappings for a 16-bit processor with separate memory and port addresses

Memory Address Map

Start	End	Description
0x0000	0x000F	Exception Vector
0x1000	0x3FFF	Program memory
0x4000	0xFFFF	Data memory

Port Address Map

Start	End	Description
0x0000	0x0000	LEDs
0x0001	0x0001	Pushbuttons
0x0002	0x000F	Unused
0x0010	0x0011	UART control
0x0012	0xFFFF	Unused

Memory Mapped IO

reminder

Peripherals accessed through a dedicated area of memory address

Advantages:

- No special instructions or techniques required
- Peripherals can be accessed with a standard C pointer

Disadvantages:

- Some address space used for peripherals
- Peripherals need complex address decoding hardware

Memory Mapped IO is the more common approach nowadays.

Example mappings for a 16-bit processor with memory and I/O on same addresses bus

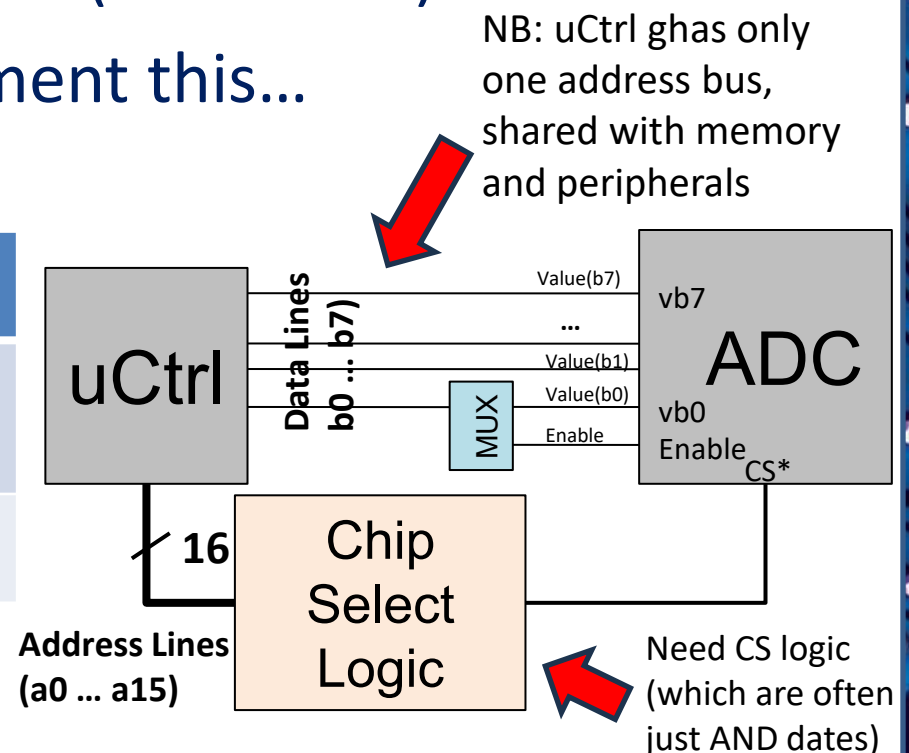
Address Map

Start	End	Description
0x0000	0x000F	Exception Vector
0x1000	0x3FFF	Program memory
0x4000	0xEFFF	Data memory
0xF000	0xFFFF	4Kb for peripherals

Memory Mapped I/O Example

- Example: assume you want to control an 8-bit ADC from a 16-bit microcontroller
- The ADC has following hardware peripheral mappings on the platform (see below)
- Multiple ways to implement this...

Address	Register Name	Description
0xFF00	Enable	Enable/disable register. Write 1 to enable device. Write 0 to disable device.
0xFF01	Value	Input value converted from the ADC



The **volatile** keyword: essential ingredient for memory mapped I/O

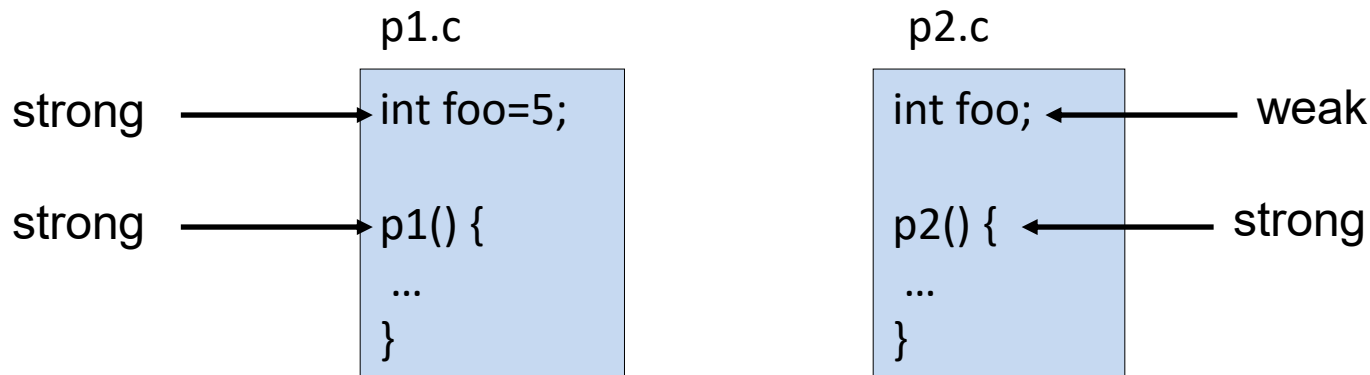
- Generally, a compiler makes use of registers where possible instead of incurring the latency of accessing memory
- When reading or writing hardware registers, you want to force the compiler to generate code that explicitly accesses the memory locations concerned
- This can be done using the volatile keyword

E.g.: The first line below may optimize the generated machine code so that it writes once to address 0xFF00 and does not bother with the read. In the second line the volatile keyword in the declaration ensures 0xFF00 is written and then explicitly read in the printf.

```
1.  int *x = 0xFF00;  x[0] = 1; printf("%d",x[0]);  
2.  volatile int *x = 0xFF00;  x[0] = 1; printf("%d",x[0]);
```

Strong and Weak Symbols

- Program symbols are either **strong** or **weak**
 - *Strong* = procedures and initialized globals
 - *weak* = uninitialized globals



Strong symbols are more **certain to be allocate addresses and spaces** in the linker file and given explicit initial values, whereas **weak symbols might get optimized out**.

Linker Sections

Supplementary.
Not examined

.text section

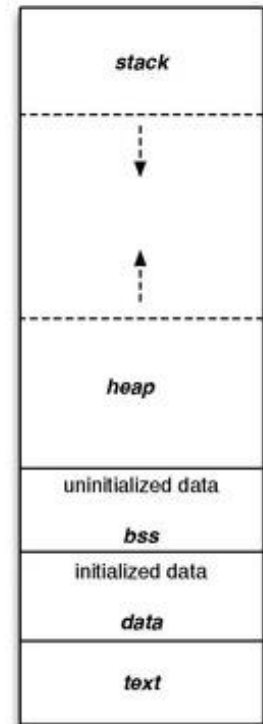
- Executable instructions and read-only fixed size constants. (i.e. not just instructions)
 - e.g. `void dostuff () { printf("hello\n"); }`
would include `printf` and the constant string in the `.text` section.

.data section

- Initialized global or static variables and memory blocks / arrays
 - e.g. `char string[] = "Hello World";`
This 12 character string and its initial characters would be slotted into the `.data` section

.bss section

- Uninitialized data section, usually adjacent to data segment. Contains all global variables and static variables that do not have explicit initialization.
 - e.g. `static int i;` would go into the BSS segment.



Memory model
for ELF-based
systems

Memory Check

Supplementary.
Not examined

1. If you initialize a global array
e.g. `int x[100] = { 0, 1, 2, 3, 4 }; ...`
which segment will this go into?

Answer: `.data section`

2. If you declare a constant variable
e.g. `const float pi = 3.141592;`
which segment will this go into?

Answer: `.text section`

3. If you declare an uninitialized counter value
e.g. `int count;`
which segment will this go into?

Answer: `.bss section` (but might get optimized out)



Memory Mapped I/O Approaches ...
Choose your favourite flavour

Memory Mapped I/O Approach 1

- Example to:
 - Creating array in C (force as strong symbol using extern)
 - Configuring the linker to place the array at a specific address

File: adc.c

```
unsigned char adc[2];
```

File: main.c

```
extern unsigned char adc[];
```

```
void main () {
```

```
    // byte to store input value
```

```
    unsigned char x;
```

```
    // enable the ADC
```

```
    adc[0] = 1;
```

```
    // read a value
```

```
    x = adc[1];
```

```
}
```

Mod. linker file

to something like:

...

```
SECTIONS {
```

...

```
# Add command lines:
```

```
ioout 0xFF00: {
```

```
    adc.o } # add all of adc.o
```

```
}
```

Memory Mapped I/O Approach 2

- Example data used in assembly that is accessible in C
 - Creating an array in assembler
 - Use extern in C to link to it

File: main.c

```
extern unsigned char adc[];  
void main () {  
    // byte to store input value  
    unsigned char x;  
    // enable the ADC  
    adc[0] = 1;  
    // read a value  
    x = adc[1];  
}
```

Assembly module:

```
...  
.org 0xFF00  
.global adc  
adc:  # use a label  
.byte adc_enable  
.byte adc_val
```

Don't worry about this too much now, we will revisit assembly next term

Memory Mapped I/O Approach 3

- Using a pointer in C
- Example:

File: main.c

```
unsigned char adc* = 0xFF00;  
void main () {  
    // byte to store input value  
    unsigned char x;  
    // enable the ADC  
    adc[0] = 1;  
    // read a value  
    x = adc[1];  
}
```

Clearly this is the approach is more likely to be used (if there are few ports)... but if you aren't using C or have a lot of assembly code that needs to access port registers then it may well be necessary to know how the other approaches work.

Memory Mapped I/O Approach 4 (ultimately my preferred approach!)

- Using a struct in C – keep it neatly together
- Example: This is technically called a peripheral register structure

File: main.c

```
typedef struct {  
    unsigned char enable; // first register at 0xFF00  
    unsigned char value;  // second register at 0xFF01  
} ADC;  
  
volatile ADC* adc = (ADC*)0xFF00;  
  
void main () {  
    // byte to store input value  
    unsigned char x;  
    // enable the ADC  
    adc->enable = 1;  
    // read a value  
    x = adc->value;  
}
```

What Approach does STM HAL use??

We can check by opening up something like “stm32f0xx_ll_gpio.h”

```
/** @brief LL GPIO Init Structure definition */
typedef struct {
    uint32_t Pin; /*!< Specifies GPIO pins to be configured.
        This parameter can be any value of @ref GPIO_LL_EC_PIN */

    uint32_t Mode; /*!< Specifies the operating mode for the
        selected pins. This parameter can be a
        value of @ref GPIO_LL_EC_MODE.
        GPIO HW configuration can be modified ...*/

    ....

} LL_GPIO_InitTypeDef;
```

So, it is Memory Mapped I/O Approach 4, using a C struct.



That stuff is in reality mucho importante!

A black and white line drawing of a stick figure standing and looking up at a thought bubble. The figure has a small circle above its head connected to the bubble.

Simon's fond reminiscence of peripheral
register structures and other useful
embedded programming methods...



And voila EEE3096S
students are happy
experts in peripheral
registers! Yaay!

But let's first confirm the
hypothesis...

Class Activity & take-home [group] exercise

Enrichment activity.
Optional. Not examined

Consider you are developing a simple digital recording device based on a 32-bit microcontroller. The peripherals and memory used by system are...

Peripherals:

- 10-bit ADC (for recording voice)
 - Has a 1 bit Data_Request line and a 10 bit Data_Out line
- USB (for downloading the recorded data)
 - Send_Data input (when set sends Data In out the USB port)
 - Receive_Data input (when set sets Data Out to last received 8bit sequence)
 - 8-bit DataIn line and 8-bit DataOut line
- 4 x LEDs (“power”, “record”, “full”, “comms”)
- 2 pushbuttons (“record/stop”, “pause/continue”)

Memory:

- 32Kb internal program flash fixed at address 0x0 – 0x7FFF
- 32Kb internal SRAM for data memory fixed at address 0x8000 – 0xFFFF
- 2 Megabytes external RAM on memory bus

TODO: Assume you want to use memory mapped port access. Develop a memory map, showing how the memory devices will be mapped and showing an appropriate mapping of where the above peripherals could be situated in the map. Note that you need to think about the chip select that would be needed so that when you read/write a particular address the correct device will be activated.



The Next Episode...

Lecture L12

Finite State Machines